

Exploring LLMs so our customers don't have to

Emil Lindfors | August 11, 2026



“Which LLM are we allowed to use?” A customer asked us that this spring, and by mid-summer it was arriving every few weeks. Here is what the same customers were doing the day before they asked:

Most of them now just use Claude Code or Cortex etc and haven’t thought about security, GDPR or cost.

That is not a criticism. Norwegian businesses have been slow to catch up on this, and the tools were good enough that nobody needed to think. Someone installed Claude Code in March, it worked, and by June the whole team used it. Nobody chose a model vendor. The laptop chose it for them.

So it’s a knowledge problem, not a procurement problem, I will say.

At work we try to run this kind of experiment before our customers need the answer. So through the spring we pointed a coding agent at every kind of endpoint we could find. We forked the agent when it would not cooperate, measured what it cost, and read a lot of GDPR guidance. This post is what we learned, plus one lens from innovation theory

that explains why the customers ended up where they did. The code is in [a companion post](#), and the cost numbers have [a post of their own](#).

Two decisions hiding in one question

Every discussion we had about this went in circles. Then we noticed there were two decisions in the room, with different people in charge of each.

	Internal consultant tooling	Client deliveries
What it is	How our own people use models on their own machines	What we recommend, build and operate for someone else
Who decides	We do	The client's legal department
What it costs	Noise against billable hours	Part of a commercial model
What binds us	Being able to say, in one sentence, what our consultants may send where	An approved endpoint and an audit trail

The regulatory frame is the same in both columns. The threat model, the cost profile and the person who has to say yes are all different. Answer both columns at once and you get the circular meetings. Answer them one at a time and most of the rest of this post follows.

Where the data goes

Three unrelated investigations landed on the same fact. A Power BI integration, a data-warehouse agent and some sandboxing work all came down to one variable: the LLM endpoint the tool was pointed at. The choice of tool barely mattered.

Here is the mechanism. A local integration on your laptop respects row-level security and role-based access control, because it is talking to the database as you. Good. But whatever the tool returns goes into the conversation context, and the context is sent to whichever model provider is configured. Microsoft says this plainly in their own documentation for the Power BI MCP server. “It runs locally” is necessary and nowhere near sufficient.

Once you accept that, the highest-value control in the whole stack is getting the client to approve a named endpoint. Everything else becomes a preference: which IDE, which agent, which framework. That was a great conclusion, until we looked at the agents we were already running.

One line of config

Commercial coding agents ship as a bundle: one vendor's client, locked to one vendor's endpoint. For a first-party product that is a reasonable decision. For a consultancy it is the wrong shape, because the endpoint is exactly the variable our clients need to control. Much of our work happens inside a client's own environment: their Microsoft 365 tenant, their cloud account, their data platform. There the endpoint choice is not ours at all. It changes the commercial model and it changes our role under any AI management standard. A tool that can only talk to one vendor's API cannot follow us in.

So we forked one. We took OpenAI's Codex CLI, which is Apache-2.0 and a really good agent, and built a small internal fork. The fork is a rebrand, telemetry off by default, its own config directory, and a handful of patches. Upstream had dropped the older Chat Completions protocol in favour of OpenAI's Responses API. That one removal is what locks the agent to a single vendor, because nearly every other inference service in the world speaks Chat Completions. Putting it back was about a thousand lines. The [companion post](#) has the patches and a ten-minute setup.

What we got out of it is that "which model provider?" stopped being an architecture question and became this:

```
model_provider = "fraktal-openrouter"  
model = "deepseek/deepseek-v4-flash-0731"
```

Swap those two lines and the same binary drives a local model on our own GPU server, Azure OpenAI with corporate single sign-on, a European sovereign inference provider, or a model inside a client's own tenant. We can now answer a legal department with "whichever endpoint you approve" and mean it.

The model and the API are two different things

Some endpoints are closed to us for reasons that have nothing to do with quality or price. China has no adequacy decision under GDPR. DeepSeek publishes no standard contractual clauses. Italy's data protection authority blocked the service within 72 hours of review, and investigations opened in more than a dozen European jurisdictions. Chinese organisations are obliged under national intelligence law to assist state intelligence work, and no contract clause overrides that. For a consultancy handling client data, [api.deepseek.com](#) is a door that is closed.

But the model and the API are two different things. Open weights running on a European endpoint, under a European processor agreement, is a completely different compliance case from the same weights behind a Chinese API. Same matrices, different lawyers.

This is why the current generation of cheap open-weight models matters commercially and not only technically. DeepSeek V4 Flash costs on the order of \$0.09 per million input tokens and \$0.18 per million output, roughly an order of magnitude below frontier pric-

ing. It is good enough to drive an agentic coding loop all day. Serve those weights from an endpoint whose jurisdiction and retention terms your client's lawyers have approved, and you have something that did not exist eighteen months ago: a stack that is cheap and defensible at the same time.

We are not alone in thinking so. By one industry estimate, over 40% of large European enterprises plan to deploy at least one open-weight model on private infrastructure by the end of 2026.

What it costs, and why that is the wrong question

This is where an argument most easily falls apart in front of people who know the numbers, so here are the numbers.

	Figure
Cheap open-weight APIs	\$0.14–0.50 per million tokens
Break-even for one high-end GPU against those	Billions of tokens per month
Utilisation needed to beat Western frontier APIs	60–70%, sustained
Cost penalty of a node running eight hours a day	5× per useful token
Model subscription per consultant	A few hundred kroner a month, low thousands with heavy agentic use
Fully loaded senior consultant	About 1.4 MNOK a year
Tooling as a share of that	About 0.3%
Two days of real agent use, measured	\$0.184 across 5.97 M tokens, a third of them prompt-cache hits

Two things follow. First, self-hosting almost never wins on price. If you present it as a cost argument, put the utilisation assumption on the same slide, or it gets taken apart in front of you. The argument that holds up is availability and control. Second, at consultancy scale the tooling cost is noise. Optimising 0.3% is optimising the wrong thing.

The cost variable worth watching is not price per token at all. Model efficiency now moves cost-per-task more than pricing does. On one recent agentic coding benchmark the most expensive model per token was the cheapest per completed task, because it used a quarter of the tokens and half the steps. Reasoning effort works the same way: it is a cost allocation, and running everything at maximum burns budget on tasks a low setting solves just as well.

The number that surprised me was not any of those. It was how much of an internal debate this started about billing. A consultancy sells hours, and an agent that finishes

the work in fewer of them is either a discount you did not intend or a margin you have to explain.

This is also an internal debate on how we as a consultancy should bill our hours, so an efficient harness with a good model mix is a competitive advantage, so that is what I would say to other consultants.

We have not settled that debate. If you run a consultancy and have not had it yet, you will.

Explore, exploit, and who does the exploring

Here is the lens. March (1991) described a tension every organisation lives with. Exploitation is using what you already know: refining the current tool, getting faster at it, squeezing more out of it. Exploration is trying what you do not know: new tools, new vendors, new ways of working, most of which will not pan out. Exploitation pays sooner and more reliably, and that is the problem. Left alone, an organisation drifts toward exploiting whatever it has, because every hour of exploring shows up as a cost this quarter and every hour of exploiting shows up as revenue. March's phrase for what the drift produces is a *competency trap*. You get so good at one way of doing things that every alternative looks worse, because you are comparing your skilled use of the old thing against your first clumsy attempt at the new one.

Read the customers through that. A team that installed Claude Code in March and was fluent by June has exploited beautifully. The questions about security, GDPR and cost never came up, because nothing in the exploiting loop forces them up. When someone then tries an open-weight model on a European endpoint for an afternoon and finds it worse, the competency trap says that is the expected result. It tells you nothing about the model.

Tushman & O'Reilly (1996) offered the organisational fix and called it the *ambidextrous organisation*. Run exploitation and exploration in separate units, with different measures and different tempos, and hold them together at the top. Their examples are large firms with a skunkworks on one side and the core business on the other, and we are not a large firm, so the transfer is a stretch. But it names what a consultancy does for its customers. A business that has to keep business as usual running cannot also spend a quarter forking coding agents. We can, and in a sense that is what they pay us for. The exploring gets outsourced.

The theory makes three predictions about this specific case, and so far all three hold:

1. **The default tool will be whatever was easiest to install.** Not the one that was approved, because nothing was approved. Ask a customer which LLM they use and the answer is a history of who on the team tried what first.

2. **The competency gap will be mistaken for a quality gap.** “We tried the open model, it was worse” usually means the team was worse at it. Budget the learning before you judge the model.
3. **Exploration has to be budgeted or the billable hour eats it.** In a consultancy this is not a metaphor. Every hour on the fork was an hour not invoiced, and it was worth it, and it still needs a line in a budget or it stops.

Four controls that survive a vendor change

Being able to point at any endpoint is necessary and it is not compliance. Four things I would want any stakeholder to take away.

- **Localisation is not compliance.** Forcing processing into the EU solves the localisation question and none of the others. Lawful basis, the processor agreement, data subject rights and your own documentation obligations sit with the controller wherever the bytes are processed. Vendor marketing skips this one most often.
- **Pin the approved models, and name an owner.** If you rely on an aggregator’s routing controls, note that the set of EU-eligible models is dynamic. Base your posture on “whatever qualifies today” and your compliance position changes without you doing anything. Pin an explicit allow-list, treat a model change as a controlled change, and name the person who owns the list and how often it is reviewed. Without an owner and a cadence, “pinned models” is an intention.
- **Metadata is still personal data.** Even with region routing, zero-data-retention and training opt-out all enabled, aggregators keep operational metadata for billing: time-stamps, model, token counts, latency. If that can be linked to a data subject, it is in scope.
- **Pseudonymise before the call.** Minimise what the prompt does not need, tokenise direct identifiers, keep the re-identification key under your own control, and treat the model call as an operation on pseudonymised text. This is the one control that holds no matter which endpoint you end up on, and it is what an auditor expects to see. One caveat: the legal bar for true anonymisation is high, and pseudonymised data is still personal data, so this shrinks the risk without removing a single obligation.

Three claims to check before you make them

Someone in any serious client meeting will know these, and overstating one costs you credibility on everything else you say.

The claim you will hear	What is actually true
The EU AI Act requires EU-only compute for high-risk systems	It does not, for most of them. EU compute makes the audit-log obligations easier to

The claim you will hear	What is actually true
	satisfy, and that is a real advantage, but it is not a legal requirement.
Changing model or hosting breaks an ISO/IEC 42001 certificate	The certificate covers the management system, not particular models. A change triggers change management, a refreshed risk assessment and supplier due diligence, about a day of process. The standard restricts how you make and document the choice, not the choice.
The high-risk requirements land in 2026	As of writing they have slipped to late 2027, with Norwegian implementation expected around mid-2027. The prohibitions and the AI-literacy requirement are unaffected. Compliance as a sales driver is a weaker argument over the next twelve months than it looked a year ago.

Where we are

None of this was a plan. It started as curiosity about whether we could run a coding agent against models we controlled, and the useful findings came out sideways. We set out to look at security and cost, and the thing that ended up mattering most was being able to change endpoint at all.

The stack we run today:

- a small, documented fork of an open-source agent
- local models on our own hardware for anything that should not leave the building
- a cheap open-weight model on a European endpoint for the bulk of day-to-day work
- a frontier vendor's enterprise API when the task warrants it
- whatever the client has approved when we work inside their environment

One binary, one config file, one pinned model list. It is still up in the air what we will decide to standardise on, and I have stopped expecting a single answer.

It's not a one size fits all solution, some are more technical than others, so I can't expect everyone to download a forked Codex and use a self-hosted DeepSeek and get good results.

So the next problem is learning. The competency trap cuts both ways. The consultants who explored are fluent in the new stack, the ones who were busy exploiting are not, and that gap is now the thing to close. That kind of learning needs close proximity, which is

hard in a distributed consultancy, so we are building digital learning arenas for it. That is the next post in this series, once it has run for long enough to have numbers.

If you are deciding this for your own organisation right now, start with the endpoint and the owner of the model list. Everything else, including the fork, is optional.

References

- March, J. G. “Exploration and Exploitation in Organizational Learning”. *Organization Science*, vol. 2, no. 1, pp. 71-87, 1991. [doi:10.1287/orsc.2.1.71](https://doi.org/10.1287/orsc.2.1.71)
- Tushman, M. L., & O'Reilly, C. A. “Ambidextrous Organizations: Managing Evolutionary and Revolutionary Change”. *California Management Review*, vol. 38, no. 4, pp. 8-29, 1996. [doi:10.2307/41165852](https://doi.org/10.2307/41165852)